

Lambda Functions

Python Lambda function is known as the anonymous function that is defined without a name. Python allows us to not declare the function in the standard manner, i.e., by using the `def` keyword. Rather, the anonymous functions are declared by using the `lambda` keyword. However, Lambda functions can accept any number of arguments, but they can return only one value in the form of expression.

Syntax

lambda arguments : expression

The expression is executed and the result is returned

```
#Add 10 to argument a, and return the result:  
x = lambda a : a + 10  
print(x(5))
```

output:

15

Lambda functions can take any number of arguments

```
#Multiply argument a with argument b and return the result  
x = lambda a, b : a * b  
print(x(5, 6))
```

output:

30

```
#calculate sum of argument a, b, and c and return the
result:
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

output:

13

Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function. Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

Syntax

```
def myfunc(n):
```

```
    return lambda a : a * n
```

```
def myfunc(n):
    return lambda a : a * n
mydoubler = myfunc(2)
print(mydoubler(11))
```

output:

22

Or, use the same function definition to make a function that always triples the number you send in:

```
def myfunc(n):  
    return lambda a : a * n  
mytripler = myfunc(3)  
print(mytripler(11))
```

output:

33

or, use the same function definition to make both functions, in the same program:

```
def myfunc(n):  
    return lambda a : a * n  
  
mydoubler = myfunc(2)  
mytripler = myfunc(3)  
  
print(mydoubler(11))  
print(mytripler(11))
```

output:

22

33

Built-in Functions

The Python built-in functions are defined as the functions whose functionality is pre-defined in Python. The python interpreter has several functions that are always present for use. These functions are known as Built-in Functions. There are several built-in functions in Python

1. abs()
2. round()
3. pow()
4. sum()
5. min()
6. max()
7. int()
8. float()
9. str()
10. list()
11. tuple()
12. set()
13. dict()
14. iter()
15. divmod()
16. len()
17. type()
18. enumerate()
19. zip()
20. unzip()
21. map()
22. filter()
23. sorted()
24. reversed()

abs() function

The python abs() function is used to return the absolute value of a number. It takes only one argument, a number whose absolute value is to be returned. The argument can be an integer and floating-point number. If the argument is a complex number, then, abs() returns its magnitude.

```
# integer number
integer = -40
print('Absolute value of -40 is:', abs(integer))
```

output:

Absolute value of -40 is: 40

```
# floating number
floating = -40.83
print('Absolute value of -40.83 is:', abs(floating))
```

output:

Absolute value of -40.83 is: 40.83

round() function

The round() function in Python is used to round a number to a specified number of decimal places.

```
round(4.6)    # Output: 5
round(4.4)    # Output: 4
round(3.14159, 2) # Output: 3.14
round(2.71828, 3) # Output: 2.718
```

pow() function

The pow() function in Python is used to raise a number to a power (i.e., exponentiation)

```
pow(2, 3)    # Output: 8 (2^3)
```

sum() function

As the name says, python sum() function is used to get the sum of numbers of an iterable, i.e., list.

```
myvar = sum([1, 2, 4])
print(myvar)
myvar = sum([1, 2, 4], 10)
print(myvar)
```

output:

7

17

min() function

The min() function in Python returns the smallest item from a set of arguments or from an iterable like a list or tuple.

```
min([4, 2, 9, 1]) # Output: 1
min("apple", "banana", "cherry") # Output: 'apple' (alphabetical order)
```

max() function

The max() function in Python returns the largest item from a set of arguments or an iterable, much like min() but in the opposite direction.

```
max([4, 2, 9, 1]) # Output: 9
max("apple", "banana", "cherry") # Output: 'cherry' (based
on alphabetical order)
```

int() function

The int() function in Python is used to convert a value to an integer.

```
int("123") #123
int(3.99) #3
```

float() function

The python float() function returns a floating-point number from a number or string.

```
# for integers
print(float(9))
# for floats
print(float(8.19))
# for string floats
print(float("-24.27"))
# for string floats with whitespaces
print(float("    -17.19\n"))

# string float error
print(float("xyz")) #error
```

output:

9.0

8.19

-24.27

-17.19

ValueError: could not convert string to float: 'xyz'

str() function

The str() function in Python converts a value into a string (text) format.

```
str(123)          # Output: '123'
str(3.14)         # Output: '3.14'

age = 25
text = "I am " + str(age) + " years old"
print(text)      # Output: I am 25 years old
```

bool() function

The bool() function in Python is used to convert a value to a Boolean (True or False).

```
bool(0)           # False
bool(1)           # True
bool("")          # False
bool("hello")     # True
bool([])          # False
bool([1, 2])      # True
```

list() function

The list() function in Python is used to create a **list** from an iterable (like a string, tuple, or set) or to create an empty list.

```
list("hello")     # Output: ['h', 'e', 'l', 'l', 'o']
list((1, 2, 3))   # Output: [1, 2, 3]
list({4, 5, 6})   # Output: [4, 5, 6] (order may vary)
list({'a': 1, 'b': 2}) # Output: ['a', 'b']
```

tuple() function

The tuple() function in Python is used to create a tuple—an immutable, ordered collection of items.

```
tuple([1, 2, 3]) # Output: (1, 2, 3)
tuple("hello")  # Output: ('h', 'e', 'l', 'l', 'o')
tuple({1, 2, 3}) # Output: (1, 2, 3) (order can vary)
```

set() function

The set() function in Python creates a set, which is an unordered collection of unique elements.

```
set([1, 2, 2, 3]) # Output: {1, 2, 3}
```

```
set((1, 2, 3, 1))
```

```
set("hello") # Output: {'h', 'e', 'l', 'o'} (order may vary)
```

dict() function

The dict() function in Python is used to create a dictionary—a collection of key–value pairs.

```
dict(name="Alice", age=30) # Output: {'name': 'Alice', 'age': 30}
```

```
dict([("a", 1), ("b", 2)]) # Output: {'a': 1, 'b': 2}
```

iter() function

The python iter() function is used to return an iterator object. It creates an object which can be iterated one element at a time.

```
# list of numbers
mylist = [1,2,3,4,5]
listIter = iter(mylist)
print(next(listIter))
# prints '1'
print(next(listIter))
# prints '2'
print(next(listIter))
# prints '3'
print(next(listIter))
# prints '4'
print(next(listIter))
# prints '5'
print(next(listIter))
```

divmod() function

Python divmod() function is used to get remainder and quotient of two numbers. This function takes two numeric arguments and returns a tuple. Both arguments are required and numeric

```
result = divmod(10,2)
# Displaying result
print(result)
```

output:

(5, 0)

len() function

The python len() function is used to return the length (the number of items) of an object.

```
mystr = 'Python'
print(len(mystr))
```

output:

6

type() function

The python type() returns the type of the specified object if a single argument is passed to the type() built in function. If three arguments are passed, then it returns a new type object.

```
mylist = [4, 5]
print(type(mylist))
mydict = {4: 'four', 5: 'five'}
```

```
print(type(mydict))
```

```
output:  
<class 'list'>  
<class 'dict'>
```

enumerate() function

The enumerate() function in Python adds a counter to an iterable and returns it as an enumerate object. This is useful when you need both the index and the value of items in a loop.

1. Basic usage with a list:

```
fruits = ["apple", "banana", "cherry"]  
  
for index, value in enumerate(fruits):  
    print(index, value)
```

```
output:  
0 apple  
1 banana  
2 cherry
```

2. With a custom starting index:

```
fruits = ["apple", "banana", "cherry"]  
  
for index, value in enumerate(fruits, start=1):#custom index  
    print(index, value)
```

```
output:  
1 apple  
2 banana  
3 cherry
```

3. Converting enumerate to a list

```
fruits = ["apple", "banana", "cherry"]
enumerated_fruits = list(enumerate(fruits))
print(enumerated_fruits)
```

output:

```
[(0, 'apple'), (1, 'banana'), (2, 'cherry')]
```

4. Using with a string:

```
word = "hello"
for index, char in enumerate(word):
    print(index, char)
```

output:

```
0 h
1 e
2 l
3 l
4 o
```

zip function

The zip() function in Python combines multiple iterables (like lists or tuples) into tuples, where the i-th tuple contains the i-th elements from all the input iterables.

```
names = ['Alice', 'Bob', 'Charlie']
scores = [85, 90, 88]

for name, score in zip(names, scores):
    print(f"{name} scored {score}")
```

output:

Alice scored 85

Bob scored 90

Charlie scored 88

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 90, 88]
ages = [23, 30, 25]
```

```
zipped = zip(names, scores, ages)
print(list(zipped))
```

Output:

```
[('Alice', 85, 23), ('Bob', 90, 30), ('Charlie', 88, 25)]
```

```
zipped = [('Alice', 85), ('Bob', 90), ('Charlie', 88)]
names, scores = zip(*zipped)
```

```
print(names)    # Output: ('Alice', 'Bob', 'Charlie')
print(scores)   # Output: (85, 90, 88)
```

output:

```
('Alice', 'Bob', 'Charlie')
(85, 90, 88)
```

unzip function

To unzip the values that were previously zipped, you can use the * operator (unpacking) with zip().

```
zipped = [('Alice', 85), ('Bob', 90), ('Charlie', 88)]
names, scores = zip(*zipped)
```

```
print(names)    # Output: ('Alice', 'Bob', 'Charlie')
print(scores)   # Output: (85, 90, 88)
```

output:
(*'Alice'*, *'Bob'*, *'Charlie'*)
(85, 90, 88)

map function()

The map() function is used to apply a given function to every item of an iterable, such as a list or tuple, and returns a map object (which is an iterator).

```
#map with user defined functions
a = [1, 2, 3, 4]
```

```
def double(val):
    return val*2
```

```
res = list(map(double, a))
print(res)
```

Output: [2, 4, 6, 8]

```
a = [1, 2, 3, 4]
```

```
#map with lambda functions
res = list(map(lambda x: x * 2, a))
print(res)
```

Output:

```
[2, 4, 6, 8]
```

filter() function

As a companion to map(), the filter() function takes a function and a list, and returns a subsetting list of the elements where the function returns true. For example, given a list of strings, return a list of the strings where the length is greater than 3:

```
# Filter strings with length greater than 3
mystr = ['apple', 'and', 'a', 'donut']
print(list(filter(lambda s: len(s) > 3, mystr)))
```

output:
['apple', 'donut']

```
#Filter even numbers with functions
# Define a function that checks if a number is even
def is_even(x):
    return x % 2 == 0
```

```
# List of numbers
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(is_even, numbers)
```

```
# Convert the filter object to a list and print
print(list(even_numbers))
```

output:

[2, 4, 6]

```
# Filter even numbers with functions with lambda functions
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers)) # Convert the filter object to a
list to display
```

Output:

```
[2, 4, 6]
```

sorted() function

Python sorted() function is used to sort elements. By default, it sorts elements in an ascending order but can be sorted in descending also. It takes four arguments and returns a collection in sorted order. In the case of a dictionary, it sorts only keys, not values.

```
mystr = "pythonpoint" # declaring string
# Calling function
sorted1 = sorted(mystr) # sorting string
# Displaying result
print(sorted1)
```

output:

```
['h', 'i', 'n', 'n', 'o', 'o', 'p', 'p', 't', 't', 'y']
```

reversed() function

The python reversed() function returns the reversed iterator of the given sequence.

```
# for string
String = 'Java'
print(list(reversed(String)))
# for tuple
Tuple = ('J', 'a', 'v', 'a')
```

```
print(list(reversed(Tuple)))  
# for range  
Range = range(8, 12)  
print(list(reversed(Range)))  
# for list  
List = [1, 2, 7, 5]  
print(list(reversed(List)))
```

output:

```
['a', 'v', 'a', 'J']
```

```
['a', 'v', 'a', 'J']
```

```
[11, 10, 9, 8]
```

```
[5, 7, 2, 1]
```



smectechnologies

GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215